TECHNICAL EXECUTIVE BRIEF

PortfolioLedger

Property-finance operations with immutable financial evidence, reproducible reconciliation and a deployable review surface.

PortfolioLedger is an independent, full-stack engineering demonstration built around a practical property-administration problem: bringing incompatible provider feeds into one defensible financial record. The implementation prioritises correctness under replays, revisions and concurrent work over polished but untraceable calculations.

Python / FastAPI

PostgreSQL / SQLAlchemy

React / TypeScript

VERIFIED ENGINEERING EVIDENCE

10,000
financial events
Isolated import and replay acceptance

0
duplicate postings
Same 10,000 entries after event replay

46
PostgreSQL tests
Final release-gate record

5 / 5
public writes denied
Hosted reviewer security verification

EXECUTIVE ASSESSMENT

What was built. A React reviewer workspace, a typed FastAPI service, an independently running PostgreSQL-backed worker and versioned data services for lease history, rental postings, reconciliations, exceptions and monthly administration-fee previews.

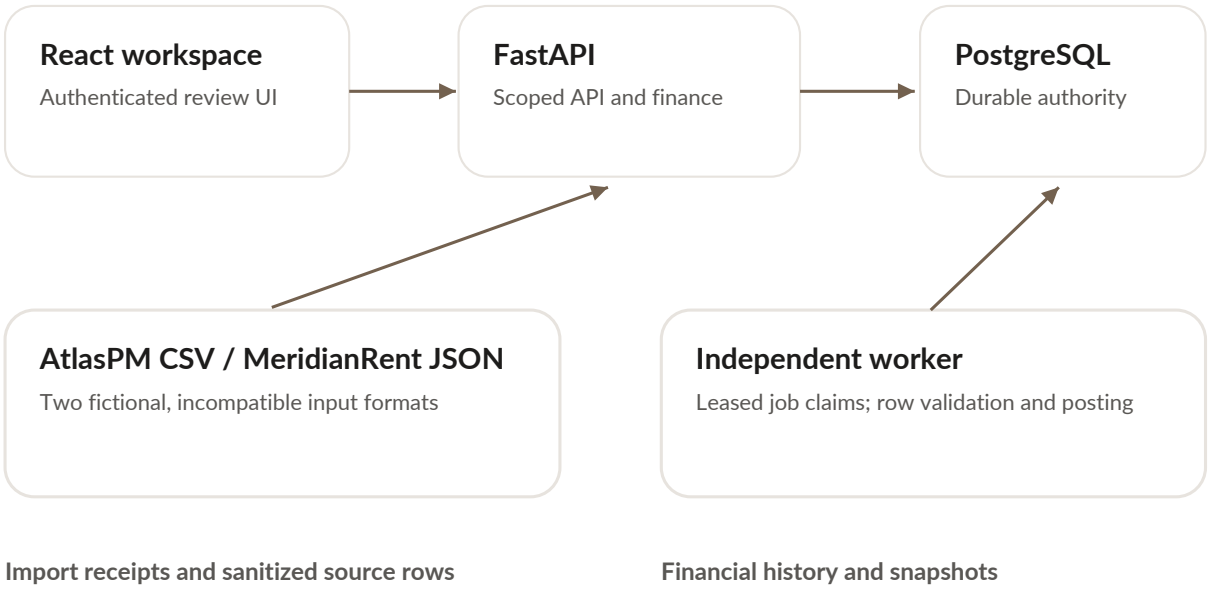
Release status: a public, password-free **read-only synthetic demonstration** is deployed on Railway with Neon PostgreSQL. Private operations and live business writes are not exposed to the public reviewer.

01 / SYSTEM DESIGN

A durable, bounded architecture.

One modular FastAPI application owns authorisation, validation and financial rules. A separate worker processes persisted imports; PostgreSQL is the single source of operational truth.

RUNTIME AND DATA FLOW



CLEAR RESPONSIBILITY BOUNDARIES

DATA INGESTION

Accept, quarantine, replay

Stable source identities, hashes, idempotency receipts, version checks and bounded retries.

FINANCIAL CORE

Preserve, compare, calculate

Append-only Decimal postings, reconciliation cutoffs, audited exceptions and fee-preview snapshots.

IMPLEMENTATION

Backend: Python 3.12, FastAPI, Pydantic v2, synchronous SQLAlchemy 2.x, psycopg 3, Alembic. **UI:** React 19, TypeScript, Vite, Lucide. **Platform:** PostgreSQL 16, Docker, Nginx, Railway, Neon, GitHub Actions.

02 / FINANCIAL INTEGRITY

Make the past inspectable.

A finance platform must survive contradictory upstream records without silently rewriting history. PortfolioLedger makes source provenance, business validity and historical snapshots explicit.

01

Append-only rental ledger

Charges increase receivables; payments and credits reduce them. A correction reverses the previous posting and creates a replacement.

02

Revision-aware ingestion

Identical revisions deduplicate. Stale revisions and changed content without revision advancement are quarantined with reasons.

03

Reconciliation with evidence

Monthly expected rent is compared with actual net source charges. Exceptions retain expected/actual values, source IDs and a cutoff.

04

Audited human decisions

A reviewer records rationale and status. Resolving a discrepancy never creates or alters rental money.

REPRODUCIBLE FEE CALCULATION

VERSION 1 / MONTH-START ELIGIBILITY

ZAR 85.00 **per lease active on the first calendar date**

No proration. Start date must be on or before day one; end date is inclusive. Pricing versions and a source-publication cutoff identify an immutable preview. Repeating the same coordinates returns the same stored result.

INVARIANTS BENEATH THE API

PostgreSQL constraints, composite portfolio foreign keys and immutability triggers reinforce application logic. Money uses **Decimal / NUMERIC(18,2)** with ROUND_HALF_UP; browser numbers format displays only. Business dates and UTC publication timestamps remain distinct.

03 / VERIFICATION AND DELIVERY

Tested beyond the happy path.

The repository records both failed checkpoints and corrected reruns. Release evidence is separated from local development results and from public hosting claims.

ISOLATED HOSTED CI ACCEPTANCE / 10K-EVENT REPLAY

87.758s	55.098s	10,000
initial import	replay	postings retained

Result: 10,002 records accepted initially (10,000 financial events + two lease rows), then 10,002 duplicates on replay; two malformed rows quarantined on each pass. Final synthetic ledger total: **ZAR 18,750,000.00**. One isolated CI run, not a production load SLO.

RELEASE SECURITY AND OPERATIONAL PROOF

ACCESS

Fail-closed public review

Password-free reviewer sessions, read-only business APIs, scoped roles, cookies, CSRF and origin restrictions.

RUNTIME

Distinct service identities

Railway web/API/worker; separate non-owner PostgreSQL grants, verified database TLS and explicit release migrations.

RECOVERY

Persistence is measured

Hosted isolated restore, worker lease recovery, full service restart and retained business IDs/amounts verified.

DELIVERY

Inspectable release gates

46 PostgreSQL tests in release record; browser suites; passed GitHub Actions and active 15-minute availability probe.

DEMONSTRATION BOUNDARIES

Entirely **synthetic**; no external property provider, insurer, payment processor or customer affiliation. The public deployment is read only, not a production finance system. Trial/free hosting quotas, six-hour Neon history and uninspected email-inbox delivery remain documented constraints.

SOURCE CODE LIVE READ-ONLY DEMO RELEASE EVIDENCE